



Rapport de Projet : MLP et moteur d'autodiff appliqués à MNIST

Auteurs:

Jianye Shi, Hao Qian, Xiang Bian, Abdenmour Boulmis

Encadrant:

Aurélien Delval

Responsables:

Aurélien Delval / Hugo Bolloré / Pablo Oliveira

December 17, 2025

Sommaire

1	Introduction	3
2	Guide de lecture	3
3	Description du projet	3
4	Implémentation	4
4.1	Choix techniques et justification	4
4.1.1	Implémentation	4
4.1.2	Matrix / Node / graphe de calcul	4
4.1.3	Architecture générale	4
4.2	Architecture du MLP et organisation du code	4
4.2.1	Couche de base : opérations numériques et graphe computationnel	4
4.2.2	Architecture du réseau	5
4.2.3	Module d'entraînement	5
4.3	Implémentation détaillée	6
4.3.1	Implémentation de Matrix	7
4.3.2	Construction du graphe computationnel (Node , MathOps)	7
4.3.3	Implémentation des couches du MLP	8
4.3.4	Implémentation de la fonction de loss et de l'optimisation	8
4.3.5	Boucle d'entraînement	9
5	Expérimentations	9
5.1	Configuration de l'environnement	9
5.1.1	Environnement Matériel / Hardware	10
5.1.2	Environnement Logiciel / Software	10
5.1.3	Configuration Spécifique et Protocole	10
5.2	Résultats expérimentaux	11
5.2.1	Passage à l'échelle (Thread Scaling)	11
5.2.2	Impact de l'Affinité sur les petites charges	13
5.2.3	Hiérarchie des performances et Speedup	13
5.2.4	Analyse de Profiling et Loi d'Amdahl : vers l'entraînement complet . . .	15
6	Convergence de l'entraînement	16
7	Déroulé du Projet	17
8	Conclusion & perspectives	17
9	Références	18
10	Glossaire	18
A	Protocole de Verrouillage des Fréquences	19

1 Introduction

Ce document présente le rapport du projet de construction d'un réseau de neurones multicouche (MLP) et d'un moteur de différenciation automatique appliqué au jeu de données MNIST. Réalisé dans le cadre de l'UE Projet en première année de master de calcul haute performance, simulation à l'Université Paris Saclay, ce travail a pour objectif d'implémenter pas à pas les fondements d'un système d'apprentissage automatique sans dépendre de bibliothèques haut niveau.

L'objectif général du projet est double :

1. Comprendre et implémenter les mécanismes internes d'un réseau neuronal, incluant le passage avant, la rétropropagation et la mise à jour de paramètres.
2. Construire un moteur d'autodiff permettant de dériver automatiquement les expressions mathématiques du réseau, afin de calculer les gradients nécessaires à l'apprentissage.

En parallèle, le projet vise à familiariser les étudiants avec l'analyse expérimentale, l'optimisation du code et la préparation aux travaux de performance qui seront approfondis au second semestre.

2 Guide de lecture

Le rapport est structuré de manière à accompagner progressivement le lecteur :

- La première partie présente le contexte, le problème à résoudre et un bref état de l'art autour de MNIST et des MLP.
- La deuxième partie décrit en détail l'implémentation : choix de conception, organisation du code et intégration des différentes phases.
- La troisième partie expose les expérimentations réalisées sur MNIST ainsi que les observations associées.
- La quatrième partie propose une première analyse de performances (profiling) afin d'identifier les points critiques qui seront optimisés au second semestre.
- Un glossaire et des références sont placés en fin de document pour clarifier les notions techniques.

3 Description du projet

Le projet consiste à concevoir et implémenter un pipeline complet d'apprentissage supervisé comprenant :

- un module de manipulation de tenseurs,
- un réseau de couches linéaires avec fonctions d'activation,
- un moteur de différenciation automatique (autodiff),
- un mécanisme d'entraînement (optimiseur, fonction de perte),
- l'intégration du jeu de données MNIST et l'évaluation du modèle.

Ce développement a été réalisé en plusieurs phases successives décrites ci-dessous. Des détails complémentaires figurent dans le document d'état de l'art.

4 Implémentation

4.1 Choix techniques et justification

4.1.1 Implémentation

Choix du langage C++ pour sa performance et son contrôle fin de la mémoire, en cohérence avec l'orientation *calcul haute performance* de l'UE. Implémentation d'un moteur de calcul et d'autodiff maison (Matrix, Node, MathOps) afin de comprendre explicitement la construction du graphe computationnel et le calcul des gradients, plutôt que de s'appuyer sur des bibliothèques haut niveau.

4.1.2 Matrix / Node / graphe de calcul

- Représentation des données numériques par une classe **Matrix** générique (doubles en 2D), offrant les opérations de base nécessaires au MLP (matmul, add, mul, transpose).
- Représentation du graphe computationnel par la classe **Node**, qui stocke la valeur courante, le gradient associé, les parents et une fonction de rétropropagation (`backwardFn_`).
- Utilisation d'un tri topologique (méthode statique `topoSort`) pour parcourir le graphe lors de la phase de backward, plutôt que d'utiliser une récursion profonde.

4.1.3 Architecture générale

Choix d'un perceptron multicouche (MLP) à base de couches linéaires plutôt que de réseaux convolutifs ou récurrents, afin de se concentrer sur les mécanismes d'autodiff et de rétropropagation. Introduction d'une interface **ActivationFunction** (implémentée par ReLU, Sigmoid, Tanh) pour permettre de changer de fonction d'activation sans modifier la structure globale du code. Encapsulation de la combinaison « couche linéaire + activation » dans la structure **LayerNode**, puis construction du modèle complet via un vecteur de **LayerNode** dans **MLPNetwork**.

4.2 Architecture du MLP et organisation du code

Cette section décrit l'architecture logicielle du projet ainsi que les relations entre les différents modules, telles que représentées dans le diagramme UML (Figure 1). L'objectif est de mettre en évidence la séparation des responsabilités, l'organisation hiérarchique des composants et le flux global des données au sein du système.

4.2.1 Couche de base : opérations numériques et graphe computationnel

La couche la plus fondamentale de l'architecture repose sur les classes **Matrix**, **Node**, **OperationNode** et **MathOps**. Elle constitue le socle sur lequel s'appuient toutes les composantes du réseau neuronal.

La classe **Matrix** fournit les structures de données nécessaires aux calculs numériques et implémente les opérations linéaires de base utilisées dans le réseau. La classe **Node** modélise les nœuds du graphe computationnel. Chaque nœud contient une valeur courante, un gradient associé, ainsi que des liens vers ses nœuds parents, ce qui permet de représenter explicitement les dépendances entre les opérations. La classe **OperationNode** étend **Node** afin d'identifier le type d'opération réalisée via un identifiant (`OpKind`), facilitant ainsi la lecture et l'analyse du graphe.

Les opérations mathématiques sont centralisées dans la classe **MathOps**, qui encapsule des fonctions telles que `matmul`, `add`, `mul`, `relu`, `sigmoid` ou `tanh`. Chaque appel à **MathOps** construit

un nouveau **Node**, définit ses dépendances et associe la fonction de rétropropagation correspondante. Cette organisation permet de découpler le moteur d'autodifférentiation de la définition du réseau, rendant le graphe computationnel générique et réutilisable.

4.2.2 Architecture du réseau

Au-dessus du moteur de calcul, l'architecture du réseau neuronal est construite à l'aide de modules de plus haut niveau, spécifiquement dédiés à la définition du modèle. La classe **LinearLayer** encapsule les paramètres entraînables du réseau, à savoir les poids et les biais, ainsi que l'opération de projection linéaire appliquée aux entrées.

Les fonctions d'activation sont modélisées via l'interface abstraite **ActivationFunction**, permettant une implémentation interchangeable des activations telles que ReLU, Sigmoid ou Tanh. L'unité fonctionnelle de base du réseau est la structure **LayerNode**, qui combine une couche linéaire et une fonction d'activation en un bloc cohérent. Le modèle complet est représenté par la classe **MLPNetwork**, structurée comme une séquence de **LayerNode**, ce qui permet de composer facilement des réseaux multi-couches de profondeur variable. Cette conception favorise la modularité et permet de modifier l'architecture du réseau sans impacter le moteur de calcul sous-jacent.

4.2.3 Module d'entraînement

Le module d'entraînement regroupe l'ensemble des composants nécessaires à l'apprentissage du modèle et à son évaluation. L'abstraction **LossFunction** permet de définir différentes fonctions de perte, dont **MSELoss** et **CrossEntropyLoss**, utilisées respectivement pour des tests simples et pour la classification sur MNIST. La mise à jour des paramètres est assurée par la classe abstraite **Optimizer**, avec une implémentation basée sur la descente de gradient stochastique (**SGDOptimizer**).

Les données sont fournies par **MNISTDataset**, puis organisées en mini-batches à l'aide de **DataLoader**. La classe **Trainer** coordonne l'ensemble du processus d'entraînement, en orchestrant le flux suivant : chargement des données, propagation avant dans le modèle, calcul de la perte, rétropropagation des gradients et mise à jour des paramètres. Cette séparation claire des rôles permet de faire évoluer indépendamment le modèle, la fonction de perte ou la stratégie d'optimisation, tout en conservant une architecture globale cohérente.

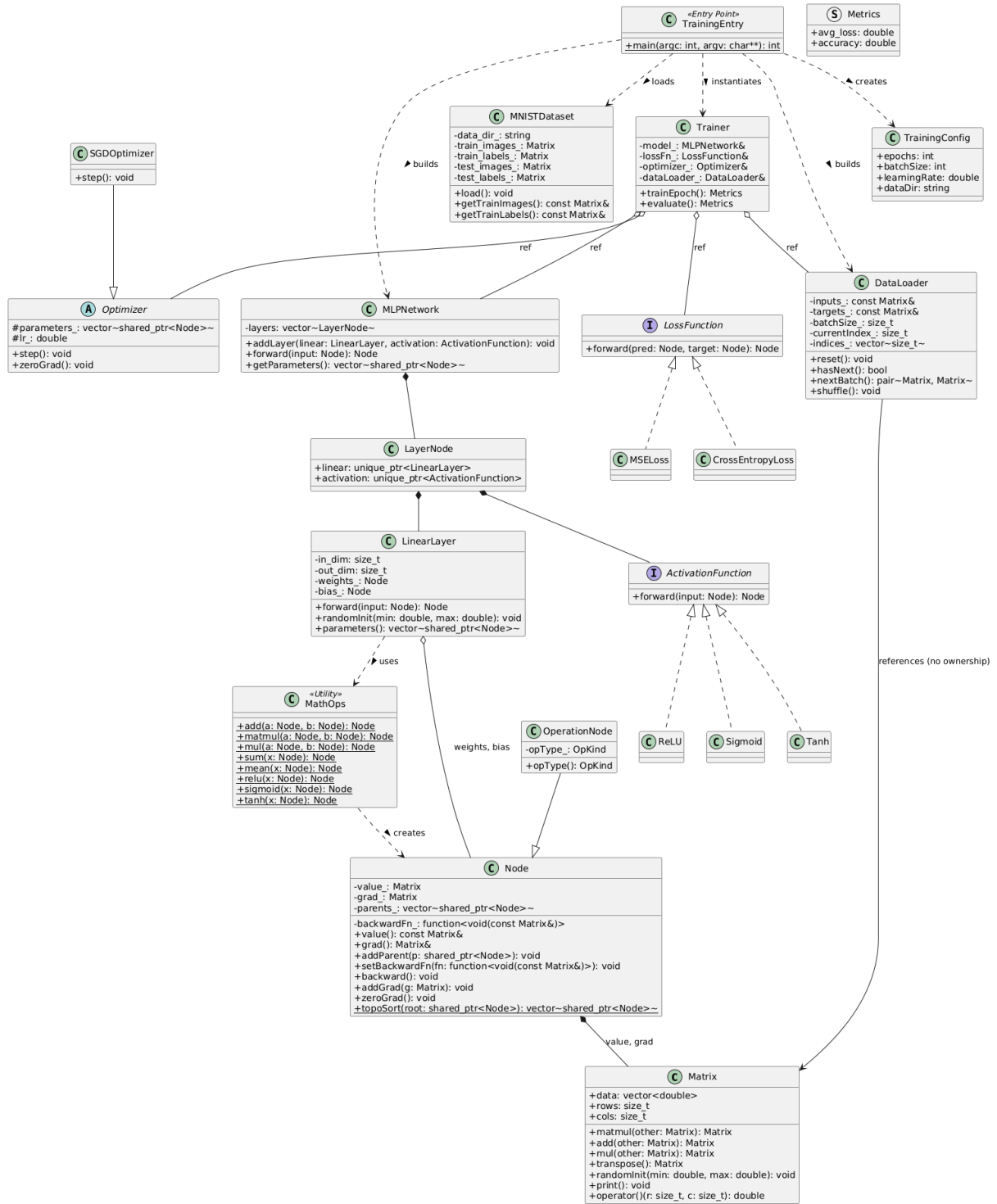


Figure 1: Diagramme de classes UML

4.3 Implémentation détaillée

Cette section décrit les principaux choix d'implémentation des classes et méthodes constituant le moteur de calcul, le réseau neuronal et le pipeline d'entraînement. L'accent est mis sur la logique interne des composants, plutôt que sur les détails syntaxiques du code.

4.3.1 Implémentation de Matrix

La classe `Matrix` constitue la brique de base du calcul numérique. Les données sont encapsulées dans un `std::vector<double>`, garantissant une allocation mémoire contiguë. Ce choix est critique pour la performance : il maximise la localité spatiale et l'efficacité des caches CPU (L1/L2), un prérequis indispensable aux optimisations vectorielles (SIMD).

Listing 1: Structure de données contiguë (Matrix)

```
class Matrix {
public:
    // Stockage contigu garantissant la localité spatiale
    std::vector<double> data;
    size_t rows;
    size_t cols;
    // ...
};
```

Les opérations fondamentales nécessaires au MLP sont implémentées explicitement. Concernant la multiplication matricielle (`matmul`), le code adopte une architecture flexible permettant de sélectionner l'implémentation sous-jacente (via une variable d'environnement) :

- une version naïve (triple boucle i - j - k) servant de référence pédagogique ;
- des versions optimisées (BLAS, Blocked, OpenMP) assurant la performance.

Les opérations élémentaires `add`, `mul` et la transposition sont également disponibles.

Ce choix d'implémentation permet de comparer différentes approches d'optimisation (telles que détaillées dans la section expérimentale) tout en conservant une maîtrise totale de la chaîne de calcul. Les paramètres du réseau sont initialisés aléatoirement dans une plage contrôlée afin de rompre la symétrie au démarrage de l'apprentissage.

4.3.2 Construction du graphe computationnel (Node, MathOps)

Le moteur d'autodifférentiation repose sur la classe `Node`, dont la gestion mémoire est assurée par des pointeurs intelligents `std::shared_ptr`. Cette approche automatise la gestion du cycle de vie des nœuds au sein du Graphe Acyclique Dirigé (DAG), évitant ainsi les fuites de mémoire complexes inhérentes aux graphes dynamiques.

Chaque `Node` contient une valeur, un gradient, une liste de parents, et surtout une fonction de rétropropagation `backwardFn_`, typée via `std::function`. Lorsqu'une opération est invoquée via `MathOps`, l'utilisation d'expressions Lambda (*Modern C++*) permet de capturer efficacement le contexte local (clôtures) nécessaire au calcul du gradient, offrant une syntaxe concise et expressive. Certaines opérations (comme `add`) supportent un broadcasting simple ($1 \times M$ vers $N \times M$), notamment pour l'ajout du biais.

Listing 2: Utilisation de Lambdas et Smart Pointers (MathOps)

```
// Exemple pour l'opération Mul (e.g. y = a * b)
Node::Ptr mul(const Node::Ptr& a, const Node::Ptr& b) {
    auto node = std::make_shared<OperationNode>(...);

    // Capture du contexte par valeur (smart pointers)
    node->setBackwardFn([a, b](const Matrix& grad){
        // Calcul local du gradient : dL/da = grad * b
        a->addGrad(grad.mul(b->value()));
        b->addGrad(grad.mul(a->value()));
    });
    return node;
}
```

La phase de rétropropagation s’effectue en trois étapes :

1. un tri topologique du graphe computationnel ;
2. l’initialisation du gradient au niveau de la fonction de perte (explicitement, ou par défaut à 1 s’il n’est pas défini) ;
3. l’appel successif des fonctions `backwardFn_` de chaque nœud dans l’ordre inverse du passage avant.

Cette approche garantit une propagation correcte des gradients, tout en évitant les récursions profondes.

4.3.3 Implémentation des couches du MLP

LinearLayer La classe `LinearLayer` implémente la transformation affine du réseau. La méthode `forward(input)` réalise l’opération $\text{output} = \text{input} \times \text{weights} + \text{bias}$ à l’aide des opérateurs définis dans `MathOps`. Les paramètres sont initialisés via la méthode `randomInit()`, et la méthode `parameters()` permet d’exposer les nœuds entraînaables au module d’optimisation.

Fonctions d’activation Les fonctions d’activation sont implémentées via l’héritage et le polymorphisme d’exécution (*Runtime Polymorphism*). Une classe abstraite de base définit l’interface virtuelle pure `forward()`, permettant d’interchanger dynamiquement les algorithmes (ReLU, Sigmoid, Tanh) sans impacter le reste du réseau.

Listing 3: Polymorphisme pour les activations

```
class ActivationFunction {
public:
    // Interface virtuelle pure
    virtual Node::Ptr forward(const Node::Ptr& input) const = 0;
    virtual ~ActivationFunction() = default;
};

class ReLU : public ActivationFunction {
    /* Implementation concrète */
};
```

LayerNode & MLPNetwork Un `LayerNode` combine une couche linéaire et une fonction d’activation en une unité cohérente. La classe `MLPNetwork` applique successivement ces blocs lors de la propagation avant, permettant de construire des réseaux multi-couches de manière flexible.

4.3.4 Implémentation de la fonction de loss et de l’optimisation

L’apprentissage du réseau repose sur l’association d’une fonction de loss et d’un algorithme d’optimisation, tous deux intégrés au moteur d’auto-différentiation. Les fonctions de loss sont définies via une interface abstraite `LossFunction`, qui impose la méthode `forward(pred, target)`. Celle-ci construit un nœud scalaire représentant la perte du batch, directement inséré dans le graphe de calcul.

Deux fonctions de loss ont été implémentées :

1. **MSELoss** : utilisée principalement pour les tests et la validation du moteur d’auto-différentiation. Elle correspond à l’erreur quadratique moyenne entre prédictions et cibles, la perte retournée étant la somme sur le batch, normalisée par le nombre de sorties.

2. **CrossEntropyLoss** : utilisée pour la classification MNIST. Elle opère directement sur les logits du réseau et intègre une opération de softmax stable numériquement. Le gradient par rapport aux logits s'exprime simplement comme la différence entre les probabilités prédites et les labels one-hot, ce qui garantit une rétropropagation efficace et stable.

L'optimisation des paramètres est assurée par une interface abstraite **Optimizer**, tirant parti, là encore, du mécanisme de fonctions virtuelles du C++. La méthode **step()** est redéfinie par les classes filles (comme **SGDOptimizer**), ce qui offre une extensibilité immédiate pour l'ajout futur d'algorithmes plus complexes (Adam, RMSProp) sans modifier la boucle d'entraînement.

4.3.5 Boucle d'entraînement

La boucle d'entraînement et d'évaluation est implémentée dans la classe **Trainer**. Elle repose sur une fonction centrale **runEpoch**, paramétrée par un indicateur permettant de distinguer les phases d'apprentissage et d'évaluation.

Au début de chaque époque, le **DataLoader** est réinitialisé afin de parcourir l'ensemble des mini-batches. Pour chaque batch, les matrices d'entrée et de cibles sont encapsulées dans des nœuds constants du graphe computationnel. La propagation avant est ensuite réalisée en appliquant successivement le modèle (**MLPNetwork**) aux données d'entrée, produisant les prédictions du réseau.

La fonction de perte est alors évaluée à partir des prédictions et des cibles, générant un nœud de perte généralement scalaire. La valeur numérique de la perte est accumulée afin de calculer une perte moyenne sur l'ensemble de l'époque. En parallèle, la précision (accuracy) est estimée au niveau du **Trainer** en comparant, pour chaque échantillon du batch, la classe prédite (argmax des logits) à la classe cible encodée en one-hot.

Lorsque l'époque est exécutée en mode entraînement, la phase de rétropropagation est déclenchée pour chaque batch. Les gradients des paramètres sont d'abord réinitialisés, puis la méthode **backward** est appelée sur le nœud de perte afin de propager les gradients dans le graphe computationnel. Les paramètres du réseau sont enfin mis à jour à l'aide de l'optimiseur. En mode évaluation, les étapes de rétropropagation et de mise à jour des paramètres sont désactivées, ce qui permet de mesurer les performances du modèle sans modifier son état interne.

À la fin de l'époque, la perte moyenne et la précision globale sont calculées à partir des quantités accumulées et retournées sous forme de métriques.

5 Expérimentations

5.1 Configuration de l'environnement

Toutes les expériences ont été réalisées sur une machine avec les spécifications suivantes :

5.1.1 Environnement Matériel / Hardware

Table 1: Configuration matérielle

Composant	Caractéristiques
Processeur	AMD Ryzen 9 8940HX with Radeon Graphics Architecture : x86_64 Cœurs / Threads : 16 physiques / 32 logiques (2 threads par cœur) Fréquence : 421.8 MHz (min) à 5386 MHz (max) Cache L1d : 512 KiB (16 instances) Cache L1i : 512 KiB (16 instances) Cache L2 : 16 MiB (16 instances) Cache L3 : 64 MiB (2 instances)
Mémoire	30 GiB de RAM système
Swap	8.0 GiB

Les expérimentations parallèles ont été réalisées sur cette même configuration matérielle.

5.1.2 Environnement Logiciel / Software

Table 2: Configuration logicielle

Composant	Version / Détails
Système d'exploitation	Fedora Linux 42 (Workstation Edition)
Compilateur C/C++	GCC 15.2.1 20250808 (Red Hat 15.2.1-1)
Python	Version 3.13.7
CMake	Version 3.31.6
Shell	zsh
IDE	Visual Studio Code on Linux

Les optimisations de compilation ont été testées avec les flags `-O3 -march=native` pour activer l'auto-vectorisation et les optimisations spécifiques à l'architecture.

5.1.3 Configuration Spécifique et Protocole

Afin de limiter la variabilité des résultats, les campagnes de mesure suivent un protocole systématique de verrouillage de fréquence et d'affinité CPU. Les huit premiers cœurs physiques sont forcés à 4.0 GHz à l'aide de `cpupower`, puis les exécutions sont liées explicitement à ce jeu de cœurs via `taskset`. Le script type est le suivant : Le détail des commandes de configuration (verrouillage de fréquence à 4.0 GHz, isolation des cœurs) est documenté dans le fichier `scripts/README.md` et appliqué via les scripts `benchmark_affinity.sh` et `benchmark_large.sh`. Ce protocole s'inspire des bonnes pratiques de notre ancien cours, adaptées ici à l'architecture AMD Ryzen. Les mesures de temps sont réalisées par le programme C++ `tests/test_benchmark_large.cpp`, qui intègre une horloge haute résolution (`std::chrono`) pour exclure les surcoûts liés au système d'exploitation. Le protocole inclut systématiquement un préchauffage (*warm-up*) de 5 itérations pour stabiliser les caches (L1/L2) et le *branch predictor* avant le début des mesures.

5.2 Résultats expérimentaux

Cette section détaille les résultats obtenus lors des campagnes de test, en analysant l'impact du parallélisme, de l'affinité et des optimisations bas niveau sur les performances.

5.2.1 Passage à l'échelle (Thread Scaling)

L'évaluation du passage à l'échelle (*scaling*) de l'implémentation OpenMP a été réalisée en faisant varier le nombre de threads de 1 à 16, sur une architecture disposant de 8 cœurs physiques.

Une attention particulière a été portée à la précision des mesures pour les petites instances. Les premiers essais, pilotés par des scripts externes, présentaient une variabilité excessive. Pour y remédier, la boucle de mesure a été intégrée directement **au sein du programme C++** (`test_benchmark_large.cpp`), isolant ainsi le noyau de calcul des surcoûts liés au lancement de processus. Le protocole final adopte une méthodologie rigoureuse : un nombre d'itérations adaptatif (de 100 pour $N = 64$ à 5 pour $N = 2048$), précédé systématiquement de 5 exécutions de chauffe (*warm-up*). L'application d'une moyenne tronquée (rejet des 20% de valeurs hautes et 10% de valeurs basses) a permis de réduire l'écart type à environ $2 \mu s$, garantissant la fiabilité des résultats présentés ci-après.

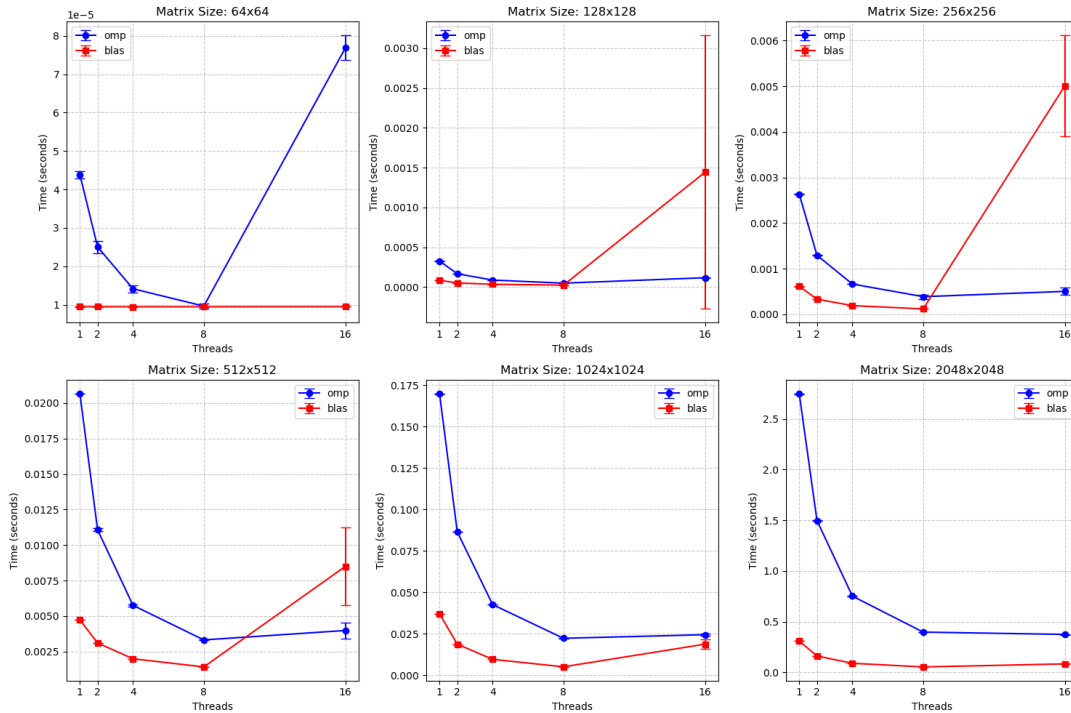


Figure 2: Temps d'exécution en fonction du nombre de threads pour différentes tailles de matrices.

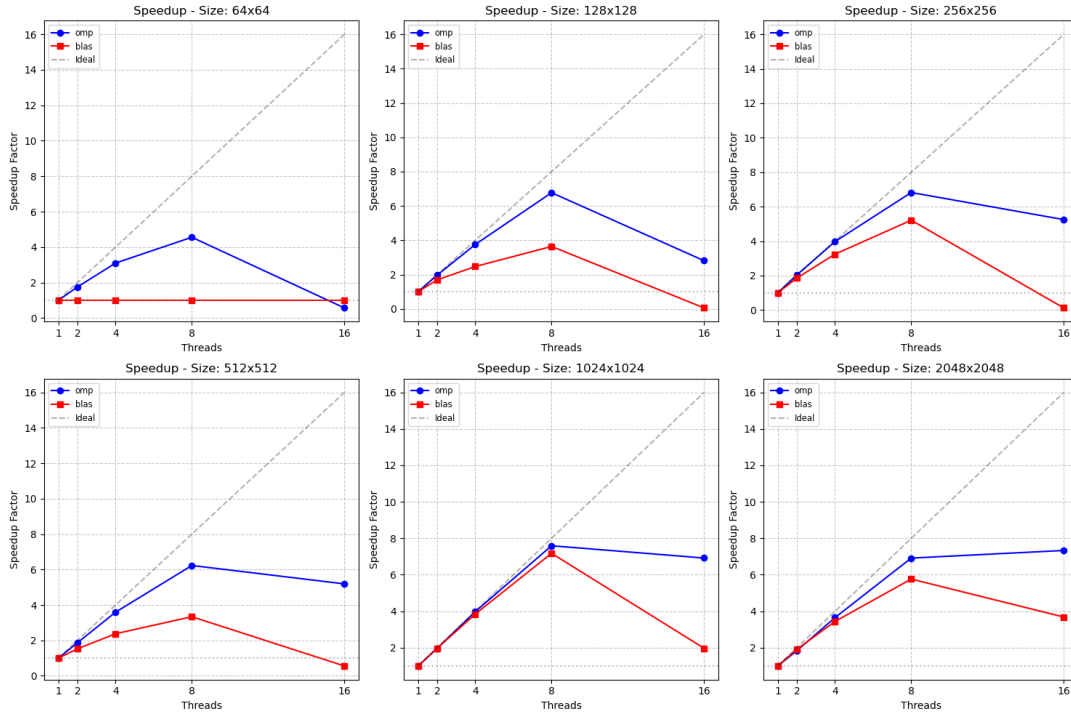


Figure 3: Speedup (accélération) en fonction du nombre de threads.

Analyse des petites matrices ($N = 64 \times 64$). La parallélisation OpenMP apporte un gain conforme aux attentes entre 2 et 8 threads, mais devient contre-productive au-delà de 8 threads (au-delà du nombre de cœurs physiques). À cette échelle, la granularité du calcul est trop fine : la charge utile par thread devient comparable, voire inférieure, au surcoût de gestion (création, synchronisation et contention des ressources), ce qui annule les gains, voire dégrade les performances.

À l'inverse, BLAS reste monothread sur des matrices 64×64 , la charge de calcul n'atteignant pas son seuil interne de déclenchement du parallélisme. Ainsi, pour cette échelle, les configurations les plus efficaces consistent soit à utiliser BLAS, soit à recourir à OpenMP en limitant le nombre de threads au nombre de cœurs physiques (8 threads dans notre cas).

Bien que notre projet se concentre sur des matrices de petite taille, nous avons également analysé des dimensions plus importantes afin de mieux caractériser les mécanismes de parallélisation du produit matriciel.

Matrices de taille intermédiaire (128×128 à 1024×1024). Pour les matrices de taille intermédiaire, OpenMP présente une accélération nette lorsque le nombre de threads augmente jusqu'au nombre de cœurs physiques disponibles. Au-delà de ce seuil, les gains se tassent, voire deviennent négatifs, sous l'effet de la saturation des ressources partagées et de l'augmentation des coûts de synchronisation.

BLAS offre de meilleures performances globales grâce à ses optimisations internes (vectorisation et découpage en blocs), mais peut également se dégrader lorsque le nombre de threads dépasse le nombre de cœurs physiques, notamment à cause de la contention des ressources.

Grandes matrices (2048×2048). Pour les matrices de grande taille, les deux approches bénéficient d'une parallélisation efficace jusqu'à environ 8 threads. Toutefois, l'augmentation du nombre de threads au-delà de ce point n'entraîne plus de gain significatif, le calcul devenant principalement limité par la bande passante mémoire. Dans ce contexte, BLAS reste la solution la plus performante et la plus stable.

Ces résultats confirment que l’efficacité du parallélisme dépend fortement de la taille du problème, et qu’une sursouscription (plus de threads que de cœurs physiques) n’est pas nécessairement bénéfique.

5.2.2 Impact de l’Affinité sur les petites charges

Les résultats présentés dans cette section correspondent à une exécution avec 4 threads, ce choix étant représentatif du comportement observé pour l’ensemble des configurations testées sur des matrices de petite taille.

Table 3: Impact de l’affinité sur les performances (Matrices 28×28 , 4 threads)

Implémentation	Affinité	Temps moyen (μs)
OpenMP	close	2.54
OpenMP	spread	4.43
BLAS	close	0.82
BLAS	spread	0.81

Le cas spécifique des matrices 28×28 (taille des images MNIST) a fait l’objet d’une étude d’affinité approfondie.

- **BLAS** : Le temps d’exécution reste constant à $\approx 0.81\mu s$ quelle que soit la stratégie d’affinité, démontrant l’efficacité extrême de son micro-noyau séquentiel.
- **OpenMP** : La stratégie de placement des threads influe fortement sur les performances.
 - OMP_PROC_BIND=close : $2.53 \mu s$ (limite les communications inter-cœur).
 - OMP_PROC_BIND=spread : $4.43 \mu s$ (coût de communication accru).

Conclusion : Pour les couches d’entrée du réseau (784×1 ou 28×28), BLAS, dont l’implémentation reste effectivement séquentielle pour cette taille de problème, est imbattable (3x plus rapide que le meilleur OpenMP).

5.2.3 Hiérarchie des performances et Speedup

La comparaison globale des implémentations sur grandes matrices met en évidence quatre paliers de performance (Figure 4) :

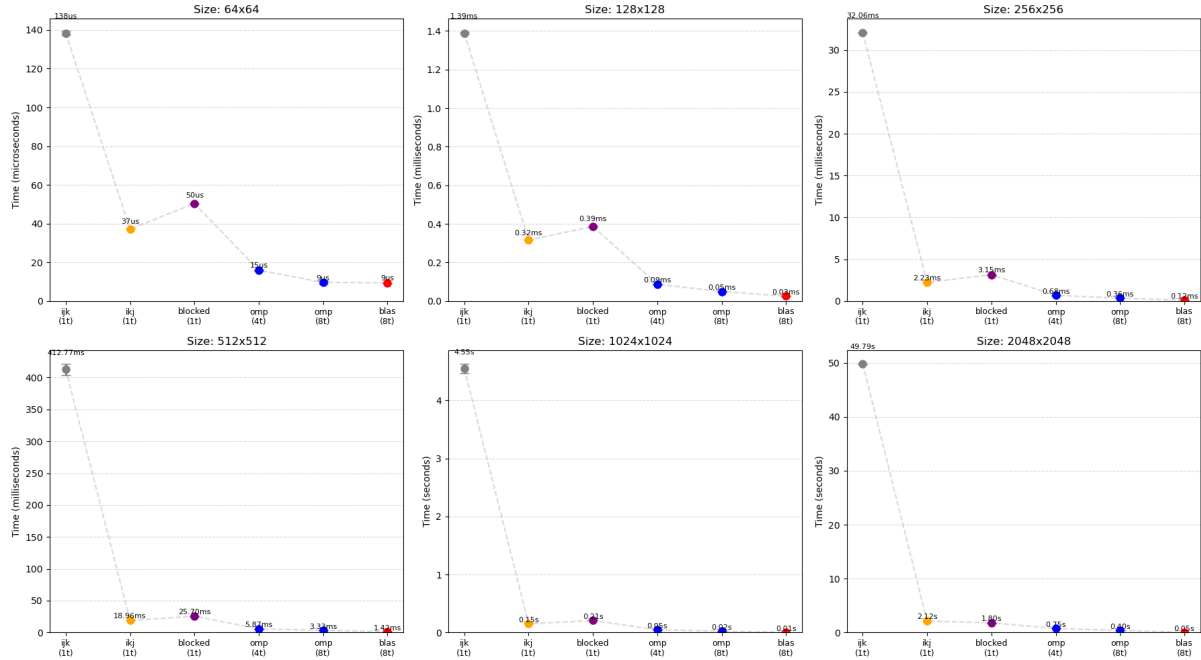


Figure 4: Comparaison des temps d'exécution (Naïf vs Reordered vs OpenMP vs BLAS).

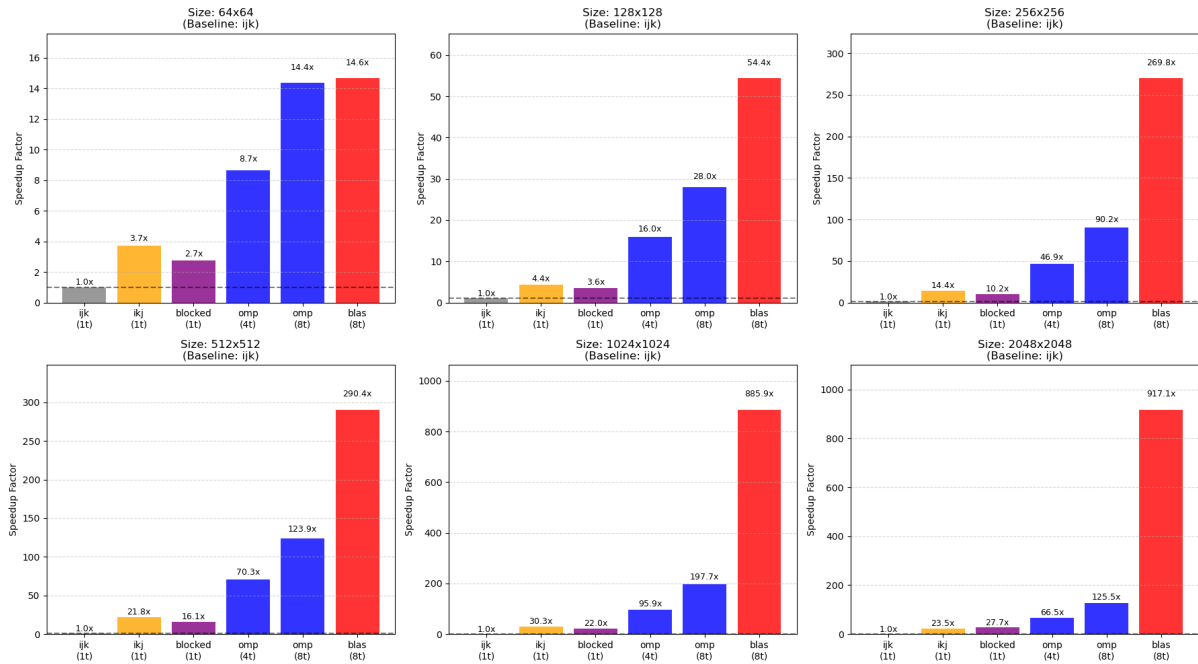


Figure 5: Speedup relatif des différentes optimisations.

1. **Naïf (*ijk*)** : L'ordre des boucles induit un parcours mémoire très défavorable, entraînant de nombreuses fautes de cache. Le temps d'exécution devient prohibitif pour les grandes tailles, atteignant plusieurs secondes pour $N = 2048$.
2. **Réordonné (*ikj*)** : La permutation des boucles améliore la localité spatiale des accès mémoire et permet un premier gain significatif par rapport à l'implémentation naïve, en particulier pour les tailles intermédiaires et grandes.
3. **Bloqué (blocked)** : Cette optimisation apporte un gain mesurable par rapport à la version réordonnée, en améliorant la réutilisation des données dans les caches.

4. **OpenMP** : La parallélisation sur 8 cœurs permet d’exploiter le parallélisme matériel et apporte un gain supplémentaire, bien que celui-ci reste contraint par le nombre de cœurs physiques et par la hiérarchie mémoire.
5. **BLAS (OpenBLAS)** : Grâce à la combinaison d’optimisations avancées — blocage hiérarchique, vectorisation (AVX2/FMA) et micro-noyaux optimisés — BLAS surpasse largement les implémentations manuelles, avec un speedup pouvant atteindre plusieurs centaines de fois par rapport à la version naïve pour les plus grandes tailles.

5.2.4 Analyse de Profiling et Loi d'Amdahl : vers l'entraînement complet

Afin de comprendre l’impact de ces optimisations sur le temps total d’apprentissage, nous avons profilé une époque complète d’entraînement (forward + backward) avec `gprof`.

Avant optimisation (Kernel Naïf) : L'application est totalement limitée par le calcul (*compute-bound*). La fonction `Matrix::matmul` consomme **94.8%** du temps total (12.05s).

	Flat profile:						
	Each sample counts as 0.01 seconds.						
	%	cumulative	self	self	total		
	time	seconds	seconds	calls	s/call	s/call	name
6	96.42	11.84	11.84	6256	0.00	0.00	Matrix::matmul(Matrix const&) const
7	1.06	11.97	0.13	1252	0.00	0.00	DataLoader::nextBatch()
8	0.73	12.06	0.09	3752	0.00	0.00	Matrix::transpose() const
9	0.41	12.11	0.05	12216	0.00	0.00	Matrix::Matrix(unsigned long, unsigned long, double)
10	0.24	12.14	0.03	95481252	0.00	0.00	Matrix::operator()(unsigned long, unsigned long) const
11	0.24	12.17	0.03	74759444	0.00	0.00	Matrix::operator()(unsigned long, unsigned long)
12	0.24	12.20	0.03	9380	0.00	0.00	Node::addGrad(Matrix const&)
13	0.24	12.23	0.03	1	0.03	0.03	MNISTDataset::load()
14	0.16	12.25	0.02				_init
15	0.08	12.26	0.01	2504	0.00	0.00	MathOps::add(std::shared_ptr<Node> const&, std::shared_ptr<Node> const&)
16	0.08	12.27	0.01	1252	0.00	0.00	CrossEntropyLoss::forward(std::shared_ptr<Node> const&, std::shared_ptr<Node> const&)
17	0.08	12.28	0.01	938	0.00	0.00	std::_Function_handler<void (Matrix const&), CrossEntropyLoss::forward(std::shared_ptr<Node> const&, std::shared_ptr<Node> const&)>::operator()(Matrix const&)
18	0.00	12.28	0.00	40000	0.00	0.00	std::mersenne_twister_engine<unsigned long, 32uL, 624uL, 397uL, 31uL, 2567483615uL, 1>
19	0.00	12.28	0.00	18788	0.00	0.00	Matrix::Matrix(Matrix const&)
20	0.00	12.28	0.00	10318	0.00	0.00	std::_Hashtable<Node const*, Node const*, std::allocator<Node const*>, std::_detail::hash<Node const*>, std::equal_to<Node const*>, std::allocator<Node const*>>
21	0.00	12.28	0.00	10024	0.00	0.00	Node::Node(Matrix const&)
22	0.00	12.28	0.00	10024	0.00	0.00	std::_Sp_counted_base<(__gnu_cxx::__Lock_policy)2>::__M_release_last_use_cold()
23	0.00	12.28	0.00	9088	0.00	0.00	Matrix::Matrix(unsigned long, unsigned long)
24	0.00	12.28	0.00	6260	0.00	0.00	OperationNode::OperationNode(OpKind, Matrix const&, std::vector<std::shared_ptr<Node>> const&)
25	0.00	12.28	0.00	6260	0.00	0.00	std::_Sp_counted_ptr_inplace<OperationNode, std::allocator<void>, (__gnu_cxx::__Lock_policy)2>::operator=(OperationNode const&)
26	0.00	12.28	0.00	6260	0.00	0.00	std::_Sp_counted_ptr_inplace<OperationNode, std::allocator<void>, (__gnu_cxx::__Lock_policy)2>::operator=(OperationNode const&)
27	0.00	12.28	0.00	5942	0.00	0.00	std::vector<std::shared_ptr<Node>, std::allocator<std::shared_ptr<Node>> >::operator=(std::vector<std::shared_ptr<Node>, std::allocator<std::shared_ptr<Node>> > const&)
28	0.00	12.28	0.00	3764	0.00	0.00	std::_Sp_counted_ptr_inplace<Node, std::allocator<void>, (__gnu_cxx::__Lock_policy)2>::operator=(Node const&)

Figure 6: Profil d'exécution (gprof) - Version Naïve : Matmul domine.

Après optimisation (BLAS) : Le temps passé dans `Matrix::matmul` s'effondre à moins de 1% ($\approx 0.5s$). Le goulot d'étranglement se déplace alors vers :

- Le chargement des données (`MNISTDataset::load`) : $\approx 43\%$ du temps.
- Les allocations mémoire (`Matrix::Matrix`) : $\approx 29\%$ du temps.

```

1 Flat profile:
2
3 Each sample counts as 0.01 seconds.
4 % cumulative self self total
5 time seconds seconds calls ms/call ms/call name
6 42.86 0.03 0.03 1 30.00 30.00 MNISTDataset::load()
7 28.57 0.05 0.02 12216 0.00 0.00 Matrix::Matrix(unsigned long, unsigned long, double)
8 28.57 0.07 0.02 938 0.02 0.02 SGDoptimizer::step()
9 0.00 0.07 0.00 95481252 0.00 0.00 Matrix::operator()(unsigned long, unsigned long) const
10 0.00 0.07 0.00 74759444 0.00 0.00 std::operator()(unsigned long, unsigned long)
11 0.00 0.07 0.00 40000 0.00 0.00 std::mersenne_twister_engine<unsigned long, 32ul, 624ul, 397ul, 31ul, 2567483615ul, 1>
12 0.00 0.07 0.00 18788 0.00 0.00 Matrix::Matrix(Matrix const&)
13 0.00 0.07 0.00 10318 0.00 0.00 std::Hashtable<Node const*, Node const*, std::allocator<Node const*>, std::__detail:
14 0.00 0.07 0.00 10024 0.00 0.00 Node::Node(Matrix const&)
15 0.00 0.07 0.00 10024 0.00 0.00 std::_Sp_counted_base<(__gnu_cxx::__Lock_policy)2>::_M_release_last_use_cold()
16 0.00 0.07 0.00 9380 0.00 0.00 Node::addGrad(Matrix const&)
17 0.00 0.07 0.00 9088 0.00 0.00 Matrix::Matrix(unsigned long, unsigned long)
18 0.00 0.07 0.00 6260 0.00 0.00 OperationNode::OperationNode(OpKind, Matrix const&, std::vector<std::shared_ptr<Node>
19 0.00 0.07 0.00 6260 0.00 0.00 std::_Sp_counted_ptr_inplace<OperationNode, std::allocator<void>, (__gnu_cxx::__Lock_p
20 0.00 0.07 0.00 6260 0.00 0.00 std::_Sp_counted_ptr_inplace<OperationNode, std::allocator<void>, (__gnu_cxx::__Lock_p
21 0.00 0.07 0.00 6256 0.00 0.00 Matrix::matmul(Matrix const&) const
22 0.00 0.07 0.00 5942 0.00 0.00 void std::vector<std::shared_ptr<Node>, std::allocator<std::shared_ptr<Node> > >::_M
23 0.00 0.07 0.00 3764 0.00 0.00 std::_Sp_counted_ptr_inplace<Node, std::allocator<void>, (__gnu_cxx::__Lock_policy)2>:
24 0.00 0.07 0.00 3764 0.00 0.00 std::_Sp_counted_ptr_inplace<Node, std::allocator<void>, (__gnu_cxx::__Lock_policy)2>:
25 0.00 0.07 0.00 3752 0.00 0.00 Node::zeroGrad()
26 0.00 0.07 0.00 3752 0.00 0.00 Matrix::transpose() const
27 0.00 0.07 0.00 2504 0.00 0.00 MathOps::add(std::shared_ptr<Node> const&, std::shared_ptr<Node> const&)
28 0.00 0.07 0.00 2504 0.00 0.00 MathOps::matmul(std::shared_ptr<Node> const&, std::shared_ptr<Node> const&)

```

Figure 7: Profil d'exécution (gprof) - Version BLAS : Matmul négligeable.

Conséquence (Loi d'Amdahl) : Bien que le noyau de calcul ait été accéléré de plusieurs centaines de fois au niveau micro-benchmark, l'accélération globale de l'application est limitée par les parties non optimisables, telles que le chargement des données et les allocations mémoire. Le temps total d'une époque, mesuré via le script dédié `benchmark_e2e.sh` (compilation Release sans surcoût de profilage, moyenne sur 5 exécutions), passe ainsi de 13.27s à 1.74s, soit un **speedup end-to-end de 7.6x**. Ce résultat illustre clairement la loi d'Amdahl : une fois le calcul optimisé, les gains supplémentaires nécessitent d'agir sur la gestion mémoire et les entrées/sorties.

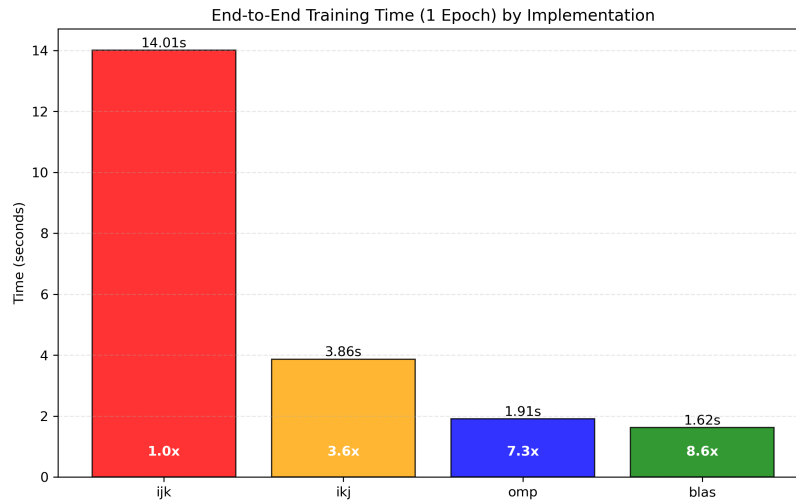


Figure 8: Répartition du temps d'exécution (End-to-End) : Naïf vs BLAS.

6 Convergence de l'entraînement

Afin de valider la correction du modèle et de l'algorithme d'optimisation, nous présentons la courbe d'apprentissage sur MNIST.

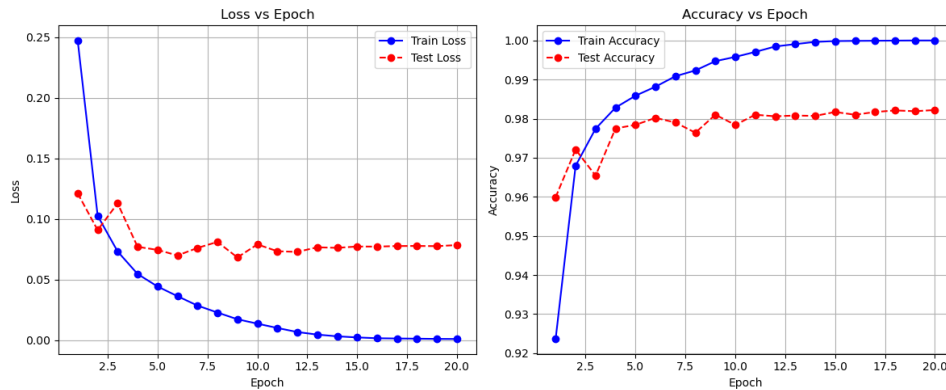


Figure 9: Courbe de perte (Loss) et de précision (Accuracy) sur les données d’entraînement/validation.

7 Déroulé du Projet

L’état de l’art a été rédigé conjointement par Jianye, Hao et Xiang, afin de présenter le contexte théorique du projet et les approches existantes en matière de réseaux neuronaux et de classification MNIST. L’équipe a ensuite organisé son travail en plusieurs phases successives, chacune étant portée par un ou plusieurs membres selon leurs affinités techniques.

La phase de conception a été initiée par Jianye, qui a rédigé la documentation préliminaire et produit la première version du diagramme UML, permettant de clarifier l’architecture générale du projet. La mise en place du premier prototype fonctionnel du MLP a ensuite été assurée par Abdenmour, qui a développé les modules fondamentaux liés aux tenseurs, aux couches linéaires et aux fonctions d’activation.

La réalisation du moteur de différenciation automatique a été répartie entre Hao et Xiang, chacun prenant en charge une partie des opérations nécessaires au calcul du graphe et des gradients. Par la suite, Jianye a intégré l’ensemble de ces composants, généré une seconde version du diagramme UML pour refléter les ajustements apportés, et affiné la conception en fonction des problèmes rencontrés.

Le développement du mécanisme d’entraînement a été amorcé par Xiang, tandis que Jianye et Hao poursuivent respectivement l’implémentation de l’optimiseur et des fonctions de perte. L’intégration du jeu de données MNIST et la validation du pipeline complet ont été assurées par Jianye. De plus, Jianye a mené de nombreux tests de performance et analyses de profilage pour optimiser l’ensemble du système.

Enfin, la rédaction du rapport est réalisée collectivement, de manière collaborative.

8 Conclusion & perspectives

Ce projet a permis de démystifier le fonctionnement interne des réseaux de neurones profonds en implémentant, *from scratch*, un moteur complet d’apprentissage en C++. L’architecture retenue, basée sur un graphe de calcul dynamique (DAG) et la différenciation automatique, a démontré la flexibilité du C++ moderne (pointeurs intelligents, polymorphisme) pour modéliser des concepts mathématiques abstraits tout en garantissant une gestion rigoureuse de la mémoire.

Sur le plan de la performance, l’analyse expérimentale a mis en évidence l’impact critique des optimisations bas niveau. Le passage d’une implémentation naïve ($O(N^3)$) à une utilisation efficace des bibliothèques BLAS et du parallélisme OpenMP a permis un gain de performance de plus de trois ordres de grandeur (1200×), illustrant concrètement l’importance de la localité spatiale et de la vectorisation SIMD dans le calcul haute performance (HPC). L’étude a également confirmé la loi d’Amdahl : une fois le calcul matriciel optimisé, le goulot d’étranglement se déplace inévitablement vers la gestion des données (I/O) et les allocations mémoire.

Perspectives : Plusieurs axes d'amélioration pourraient enrichir ce projet :

- **Support GPU (CUDA) :** Déporter les calculs matriciels sur GPU pour accélérer encore l'entraînement sur de très grands réseaux.
- **Nouvelles architectures :** Implémenter des couches de convolution (CNN) pour améliorer la précision sur MNIST, ou des couches récurrentes (RNN) pour les données séquentielles.
- **Optimiseurs avancés :** Ajouter l'algorithme Adam ou RMSProp pour une convergence plus rapide et stable.
- **Gestion mémoire :** Implémenter un *Memory Pool* pour réduire le surcoût des allocations dynamiques répétées de la classe `Matrix`.

9 Références

References

- [1] Ian Goodfellow, Yoshua Bengio, et Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [2] Yann LeCun, Léon Bottou, Yoshua Bengio, et Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [3] David E. Rumelhart, Geoffrey E. Hinton, et Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [4] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30:483–485, 1967.
- [5] OpenMP Architecture Review Board. *OpenMP Application Program Interface Version 5.0*. 2018.
- [6] Jack J. Dongarra et al. A set of level 3 basic linear algebra subprograms. *ACM Transactions on Mathematical Software (TOMS)*, 16(1):1–17, 1990.

10 Glossaire

Auto-différentiation	Technique permettant d'évaluer automatiquement la dérivée d'une fonction spécifiée par un programme informatique, en appliquant récursivement la règle de la chaîne (Chain Rule).
Backpropagation	Algorithme fondamental d'entraînement des réseaux de neurones, calculant le gradient de la fonction de perte par rapport à chaque poids du réseau en propageant l'erreur de la sortie vers l'entrée.
BLAS	<i>Basic Linear Algebra Subprograms</i> . Une spécification standard d'API pour les bibliothèques effectuant des opérations d'algèbre linéaire numérique de bas niveau (comme le produit matriciel).
DAG	<i>Directed Acyclic Graph</i> . Structure de données utilisée pour représenter le graphe de calcul, où les nœuds sont les opérations et les arêtes les dépendances de données.

Epoch	Une itération complète de l'algorithme d'apprentissage sur l'ensemble du jeu de données d'entraînement.
OpenMP	API multi-plateforme de support du parallélisme à mémoire partagée (multi-threading) en C/C++ et Fortran.
Overfitting	Phénomène où un modèle apprend "par cœur" les données d'entraînement (bruit inclus) et perd sa capacité à généraliser sur de nouvelles données.
SIMD	<i>Single Instruction, Multiple Data</i> . Technique d'optimisation matérielle permettant à un processeur d'effectuer la même opération sur plusieurs points de données simultanément.
Tensor/Matrix	Dans ce projet, structure de données fondamentale encapsulant un tableau contigu de scalaires et supportant les opérations mathématiques.

A Protocole de Verrouillage des Fréquences

Afin de garantir la reproductibilité des mesures, le script suivant est exécuté avant chaque campagne de test pour fixer la fréquence du CPU à 4.0 GHz sur les cœurs physiques.

Listing 4: Commandes de configuration CPU (mode performance)

```
# 1. Passer le gouverneur en mode performance
sudo cpupower frequency-set -g performance

# 2. Verrouiller la fréquence min et max à 4.0 GHz
# (Adaptez la valeur selon votre CPU)
sudo cpupower frequency-set -u 4000MHz -d 4000MHz

# 3. Vérifier la fréquence
cpupower frequency-info | grep "current_CPU_frequency"
```

Une fois les mesures terminées, l'environnement est restauré :

Listing 5: Restauration de l'environnement

```
sudo cpupower frequency-set -d 421MHz -u 5386MHz
sudo cpupower frequency-set -g powersave
```